

Receipt Capture Web SDK — Technical Specification

ReceiptCaptureSDK (version **0.1.0**) is a dependency-free, browser-based SDK that captures a receipt with the device camera, prepares an OCR-optimized image, and runs it through a cloud backend powered by **Amazon Textract** — returning structured receipt data (vendor, date, total, line items). It is a single JavaScript file that injects its own full-screen guided-capture UI.

How to read this document

If you are...	Read
---------------	------

A **stakeholder / lead** wanting to know what it does **Part 1 — Overview**

A **developer integrating** the SDK into an app Parts **1 + 2**

A **developer maintaining / extending** the SDK Parts **1 + 3 + 4**

This document mirrors `static/receipt-capture-sdk.js` as built (v0.1.0). Where a behavior is the integrating app's responsibility rather than the SDK's, it is called out explicitly.

Part 1 — Overview

Audience: everyone.

What it is

A single JavaScript file ([static/receipt-capture-sdk.js](#)) that you drop into a web page. Calling `captureFlow()` opens a full-screen, guided camera experience, helps the user frame the receipt, captures an optimized image (or several sections for a long receipt), sends it to a backend that extracts the contents with Amazon Textract, and resolves with the structured result.

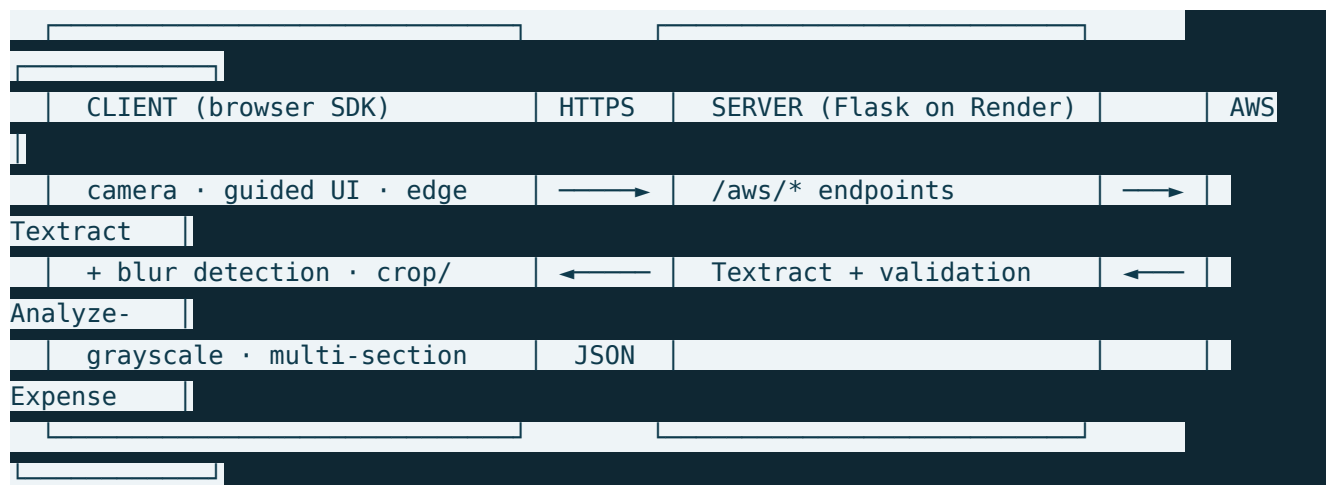
What it does

- **Guided camera capture** — rear camera, a fixed alignment box with a dimmed

surround and corner brackets, per-edge guides that turn from amber to **green** when the receipt edge is detected and aligned, live lighting/glare coaching, and a flash toggle (shown only when the device supports it). Capture is **manual** (the user taps the shutter).

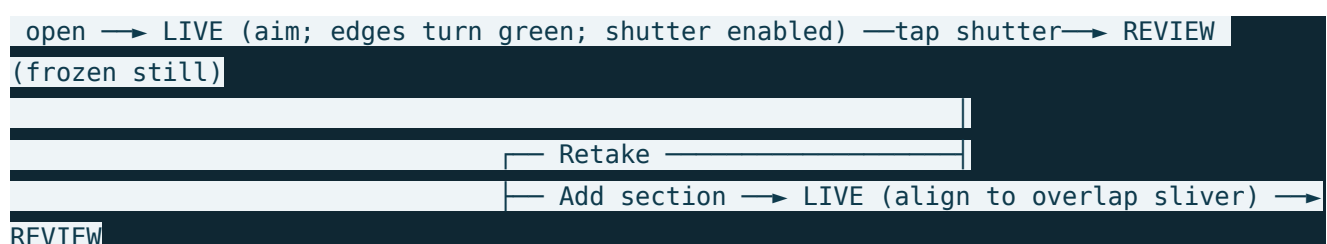
- **On-device quality checks** — brightness and glare checks, OpenCV edge detection, and a **blur (sharpness) check** so only a real, readable, in-focus receipt is accepted; a failed check coaches the user and discards the frame.
- **Long receipts** — captured in multiple sections with an on-screen overlap sliver to line up the next part; the section results are merged into one clean item list (the overlap is removed once via fuzzy matching, real repeat purchases are kept).
- **Image preparation** — crops to the alignment box, corrects rotation, converts to a compressed grayscale JPEG sized for OCR, and enforces a client-side upload size cap.
- **Resilient, secure delivery** — uploads to the backend with progress reporting, automatic retry/backoff and timeouts, then polls for the result; the browser never holds cloud credentials and talks only to the app's own endpoints.

Architecture at a glance (cloud-hybrid)



Security boundary: all OCR, validation, and (when enabled) fraud/dedup logic runs server-side. The client only prepares and delivers the image. The browser talks only to the app's own `/aws/*` endpoints — never to AWS directly.

The capture experience





Part 2 — Integration Guide

Audience: developers adding the SDK to an app.

Requirements

- **Secure context** — the camera requires `https://` (or `http://localhost`).
- A **modern browser** with `getUserMedia`. The SDK runs anywhere `getUserMedia` is available, **including desktop browsers** (Chrome/Edge/Firefox/Safari) — it does not gate itself by device type (see *Device gating*, below).
- A reachable **backend** exposing the `/aws/*` endpoints (see Part 3) with CORS allowing your page's origin.

Quick start

```
<!-- 1. Point at your backend, then load the SDK -->
<script>window.BACKEND_URL = "https://your-backend.example.com";</script>
<script src="receipt-capture-sdk.js"></script>

// 2. Launch the guided capture and use the result.
async function scanReceipt() {
  const base = (window.BACKEND_URL || "").replace(/\/$/, "");
  const sdk = new ReceiptCaptureSDK({
    userId: "member-123",
    endpoints: {
      uploadUrl: base + "/aws/get-upload-url",
      status: base + "/aws/analyze-status",
    },
  });

  sdk.on("status", s => console.log(s.phase, s.progress ?? "")); // optional progress UI

  try {
    const receipt = await sdk.captureFlow(); // opens the full-screen UI
    console.log(receipt.vendor.name, receipt.total);
    receipt.line_items.forEach(li => console.log(li.description, li.total));
```

```
// → credit cashback, match offers, etc.
} catch (err) {
  if (err.cancelled) return; // user closed the camera
  alert(err.message); // camera couldn't start
}
}
```

The result object (what `captureFlow()` resolves to)

```
{
  "vendor": { "name": "WHOLE FOODS MARKET" },
  "date": "06/13/2026",
  "total": 32.54,
  "currency_code": "USD",
  "document_type": "receipt",
  "line_items": [
    { "description": "OG HONEYCRISP APPLE", "total": 2.39, "quantity": "0.35 lb" }
  ],
  "line_item_count": 11,
  "avg_confidence": 95.38,
  "source": "aws-textract",
  "merged_sections": 2,
  "sdk_version": "0.1.0"
}
```

`merged_sections` and `sdk_version` are always present; `merged_sections` is 1 for a single-section capture.

Error handling

`captureFlow()` resolves with the result on success. Otherwise:

Outcome	How it surfaces
User closes the camera	Promise rejects with <code>{ cancelled: true }</code>
Permission denied	An in-UI recovery panel appears with per-browser instructions (Try again / Cancel); the promise also rejects with an <code>Error</code>
Camera can't start (not HTTPS, no device)	Promise rejects with an <code>Error</code>
Server rejects the receipt	Shown inside the SDK's UI (error panel with Retake /

Outcome	How it surfaces
---------	-----------------

(unreadable, low confidence, too old) Close); the user retakes or closes (→ cancelled)

So your `try/catch` only needs to handle **cancel** and **camera-start failure** — analysis rejections and permission recovery are presented in the capture UI itself.

Options

Option	Default	Description
<code>endpoints.uploadUrl</code>	<code>/aws/get-upload-url</code>	Endpoint returning a job id + PUT URL
<code>endpoints.status</code>	<code>/aws/analyze-status</code>	Poll endpoint
<code>userId</code>	<code>"demo-user"</code>	Sent as <code>X-User-Id</code> on upload (used by dedup)
<code>idealWidth / idealHeight</code>	<code>1920 / 1080</code>	Requested capture resolution
<code>jpegQuality</code>	<code>0.85</code>	Output JPEG quality (falls back to 0.6, then PNG)
<code>grayscale</code>	<code>true</code>	Convert output to grayscale
<code>lumaMin / lumaMax</code>	<code>40 / 230</code>	Luminance guardrail bounds
<code>glareMax</code>	<code>0.14</code>	Near-white fraction before a glare warning
<code>blurThreshold</code>	<code>80</code>	Min Laplacian-variance sharpness; below this a capture is discarded with a "blurry" coach
<code>edgeDetection</code>	<code>true</code>	Enable OpenCV edge detection
<code>box</code>	<code>{left:0.08, right:0.92, top:0.11}</code>	Fixed alignment box (viewport fractions)
<code>alignTol</code>	<code>0.12</code>	$\pm$ tolerance for an edge to count as aligned

Option	Default	Description
		(green)
<code>contrastMin / textureMin</code>	<code>26 / 0.035</code>	Real-receipt detection gates
<code>maxUploadBytes</code>	<code>20 MB</code>	Client-side upload size cap (rejects larger images before upload)
<code>pollIntervalMs / maxPolls</code>	<code>2000 / 60</code>	Status polling cadence / ceiling (~2 min)
<code>fetchTimeoutMs</code>	<code>30000</code>	Per-request timeout
<code>retryAttempts / retryBaseMs</code>	<code>3 / 1000</code>	Retry count and backoff base for get-URL / poll requests (5xx / 429 / timeout)
<code>fuzzyMergeThreshold</code>	<code>0.75</code>	Token-similarity threshold for multi-section overlap merge
<code>opencvUrl</code>	<code>.../opencv.js</code>	OpenCV.js source (lazy-loaded)

Methods & events

Method	Description
<code>new ReceiptCaptureSDK(options)</code>	Create an instance
<code>captureFlow() → Promise<result></code>	Open the guided UI; resolve with the OCR result
<code>on(event, cb)</code>	Subscribe to progress events (chainable)
<code>start() / stop()</code>	Start / stop the camera stream directly
<code>capture() / submit()</code>	Capture a frame / upload + analyze the captured section(s)
<code>segmentCount · clearSegments() ·</code>	Inspect / manage captured sections

Method	Description
<code>removeLastSegment()</code>	
<code>close()</code>	Tear down the UI and stop the camera

Event	Payload	When
<code>ready</code>	<code>{ width }</code>	Camera started
<code>segment</code>	<code>{ count, luminance, box? }</code>	A section was captured
<code>status</code>	<code>{ phase, message, attempt?, status?, progress?, loaded?, bytes? }</code>	Pipeline progress. Phases: <code>detector-ready</code> , <code>fallback</code> , <code>requesting-url</code> , <code>uploading</code> (with <code>progress %</code>), <code>polling</code> (with <code>attempt</code>), <code>orientation-changed</code>

Device gating (desktop vs mobile)

The SDK itself is **device-agnostic**: `captureFlow()` opens the guided camera on any browser that grants `getUserMedia`, desktop included. It does **not** restrict capture to touch devices. Whether to offer live camera capture on desktop or steer desktop users to a file **upload** instead is a **decision for the integrating app** (and an open product decision — see requirement **CAP-10**). If the app chooses to gate, `matchMedia("(pointer: coarse)")` is a reasonable touch-device test; the upload path reuses the same `get-upload-url` → `PUT` → `poll` endpoints with the chosen file as the body.

Part 3 — Technical Internals

Audience: developers maintaining or extending the SDK.

3.1 Camera constraints

Requests the rear camera with `facingMode:{exact:"environment"}` and advanced: `[{focusMode:"continuous"}]`; falls back to `facingMode:"environment"` then the default sensor (emitting a `status:"fallback"` event). Ideal resolution 1920×1080. The `<video>` is forced `playsinline/autoplay/muted` (attributes **and** properties) for iOS Safari. A debounced `resize / screen.orientation` handler recomputes geometry and re-shows the overlap sliver on

rotation.

3.2 On-device analysis loop

Every ~200 ms (while live and idle) the SDK samples a downscaled (240 px) cover-crop of the video and computes mean Rec.709 luminance and a near-white glare fraction, driving the coaching messages ("Too dark", "Too bright", "Glare — tilt the receipt"). When OpenCV is ready it also runs edge detection and lights the "ready" dot when both side edges are aligned.

3.3 Edge detection (OpenCV.js)

- **Lazy-loaded** (~8 MB WASM) from a CDN when the camera opens; falls back to the fixed guide box (no green feedback) if it fails to load or initialize (30 s cap).
- **Brightness-band** detection — the receipt is lighter than its background — not gradient-based, which would mistake interior text for an edge.
- **False-positive gates:** (1) a contrast gate (the bright band must exceed the background by `contrastMin`) and (2) a Canny **text-density** gate (`textureMin`) so smooth surfaces like walls or hands are rejected.
- **Per-edge state:** an edge is "in view" only when the band starts/ends inside the frame; it turns **green** when that edge lands within `alignTo1` of the fixed box edge. The **bottom** line appears only when the receipt's end is detected.
- All OpenCV Mats are tracked and released in a `finally` block (no leaks even on mid-function exceptions).

3.4 Image processing & quality gates

- **DPR-aware crop:** the fixed box is mapped onto the raw sensor matrix via `getBoundingClientRect()` ratios and rasterized at `devicePixelRatio`.
- **Skew correction:** landscape sensor + portrait layout → rotate the canvas 90° CW.
- **Luminance guardrail:** mean Rec.709 luminance must be within [`lumaMin`, `lumaMax`], or the capture throws "Too dark/bright" and is discarded.
- **Blur gate:** the captured section's sharpness is measured as **Laplacian variance**; below `blurThreshold` the section is removed and the user is coached to hold steady and retake.
- **Output:** grayscale JPEG at `jpegQuality`, with automatic fallback to lower quality then PNG if the browser can't encode JPEG. A client-side **size cap** (`maxUploadBytes`) rejects oversized images before upload.

3.5 Multi-section merge

Sections are **not pixel-stitched** (fragile on thermal paper). Each section is uploaded and analyzed independently, then results are merged: - The capture overlap is a contiguous run of lines shared between sections. - The merge finds where the next section's leading run matches a contiguous run **anywhere** in the accumulated items and drops that run **once**. - Matching is **fuzzy token-similarity** (Jaccard-style token overlap $\geq$ `fuzzyMergeThreshold`, default 0.75) rather than exact string equality, so OCR drift between sections still merges. - Because it matches *runs*, legitimately repeated purchases are preserved. - The merged result takes the first non-empty vendor/date, the last section's total, the averaged confidence, and stamps `merged_sections` and `sdk_version`.

3.6 Delivery pipeline

- **Get URL:** POST `endpoints.uploadUrl` → { `job_id`, `upload_url` } (with retry + timeout).
- **Upload:** PUT `upload_url` via XMLHttpRequest for **upload-progress** events, sending X-User-Id and X-SDK-Version headers.
- **Poll:** GET `endpoints.status?job=<id>` every `pollIntervalMs` up to `maxPolls`; terminal statuses are `done` (→ result), `rejected` (server declined; surfaced in the error panel), `failed`, and `unknown` (job not found).
- **Resilience:** `get-upload-url` and each poll use timeouts and exponential backoff with jitter, retrying on 5xx / 429 / timeout (`retryAttempts`); 4xx are not retried. The upload itself is a single PUT with its own timeout.

3.7 Backend API (`/aws/*`)

Flask + boto3 ([app.py](#)). The SDK client calls **get-upload-url**, the returned presigned-style **PUT**, and **analyze-status**; the others are server routes.

Endpoint	Purpose
GET <code>/aws/status</code>	Whether boto3 + AWS credentials are available
POST <code>/aws/get-upload-url</code>	Returns { <code>job_id</code> , <code>upload_url</code> } (absolute, CORS-friendly)
PUT <code>/aws/upload/<job_id></code>	Receives image bytes; starts Textract in a background thread
GET <code>/aws/analyze-status?job=<id></code>	{ <code>status</code> : <code>processing</code> <code>done</code> <code>rejected</code> <code>failed</code> <code>unknown</code> , <code>result?</code> , <code>error?</code> }

Server-side validation layer: readability hard block · confidence gate (TEXTTRACT_MIN_CONFIDENCE, default 50) · temporal rules (future-dated / older than RECEIPT_MAX_AGE_DAYS, default 14) · line-item cleanup (strips Textract phantom rows + summary/payment lines; does **not** dedup by description, so repeats survive) · deduplication (SHA-256 image hash + user_vendor_total_date semantic key) — **gated by DEDUP_ENABLED** and off during prototyping.

3.8 Code map

File	Contains
static/receipt-capture-sdk.js	The SDK: UI build, camera, _detectEdges, _captureBox, blur (_measureSharpness), submit/_analyzeBlob/_mergeResults, _fetchWithRetry
app.py	/aws/* routes, normalize_textract, run_analyze (validation), _clean_line_items
design_preview_noclip.html	Reference integration: app-level device choice, upload path, result rendering

Part 4 — Limitations & Production Roadmap

Known limitations

- **Edge/bottom detection** needs the receipt lighter than its background — best on a dark, non-shiny surface; weak on white tables. A heavier on-device ML model would improve robustness.
- **OpenCV.js is ~8 MB**, lazy-loaded on first camera open; on slow networks edge detection activates a few seconds in (manual capture still works meanwhile, using the fixed box).
- **Fuzzy overlap merge** is token-based; extreme OCR drift on a shared line could still merge imperfectly, though far less than exact-match would.
- **Textract synchronous AnalyzeExpense** supports JPEG/PNG, not PDF.

Production extension points

The current backend is a lightweight stand-in: Textract runs synchronously on the uploaded bytes (no S3), and jobs/dedup live in process memory. A production build would use **API Gateway + Lambda + an S3 presigned PUT + asynchronous Textract + DynamoDB** for the dedup indexes. The client pipeline (presigned PUT + polling) already matches that shape, so **only the server changes** — the SDK is unaffected.

Configuration (backend env)

`AWS_REGION` · AWS credentials (env or IAM role) · `ALLOWED_ORIGIN` (CORS) · `RECEIPT_MAX_AGE_DAYS` · `TEXTRACT_MIN_CONFIDENCE` · `DEDUP_ENABLED`.

Tech stack

Vanilla ES + Canvas 2D + `getUserMedia` (SDK) · OpenCV.js 4.8 (WASM, lazy) · Flask + flask-cors + boto3 + gunicorn (backend) · Amazon Textract AnalyzeExpense · GitHub Pages (frontend) + Render (backend).

SDK version 0.1.0. This specification is generated from and kept in sync with `static/receipt-capture-sdk.js`; update the code and regenerate this document together.